10/11/25

a) Create a linked list

b) Insertion of a node at first position, any position, end of the list

c) Display the contents of the linked

NAP to implement singly linked list with the following operations as above

Pseudo code:

```
struct node {
    int data;
    node *next;
};
struct node *head=NULL, *temp, *newnode.
*newnode =(struct node*) malloc (sizeof (node))
```

insertion at first position

→ create a newnode, enter the data, initise newnode

→ newnode→next = head

If newnode →null :

insertion at any position

→ create a newnode, enter the data.

→ ask user to enter the position to enter the element

→ run a for loop (i=1; i< pos-1 && temp != NULL; i++)

temp = temp →next;

→ if temp == Null

printf("Invalid position);

→ else

newnode→next = temp→next;

temp→next = newnode;

insertion at end of the list

→ create a newnode, enter the data, set newnode→next = NULL

→ if (head == Null)

head = newnode;

→ else {

struct node *temp = head

while (temp→next != NULL) {

temp = temp→next; }

```
        temp->next = newnode;
    }

# To display the contents
    struct node *temp = head;
→ if (temp == NULL)
    printf("List is empty");
→ while (temp != NULL) {
        printf("%d", temp->data);
        temp = temp->next;
    }
```

execute

```
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;
void CreateList(int n) {
    struct Node *newnode, *temp;
    int data, i;
    if (n <= 0) {
        printf("number of nodes should be greater than 0 \n");
        return;
    }
    for (i=1; i<=n; i++) {
        Newnode = (struct Node*)malloc(sizeof(struct Node));
        if (Newnode == NULL) {
            printf("Memory allocation failed \n");
            return;
        }
        printf("Enter data for node %d:", i);
        scanf("%d", &data);
        newNode -> data = data;
        Newnode -> next = NULL;
        if (head == NULL) {
            head = Newnode;
        } else {
```

```
        temp->next = Newnode;
      }
      temp= Newnode ;
    }
    printf ("\n Linked list created successfully .\n");
  }
Void insertAtBeginning (int data) {
  Struct Node * newnode = (struct Node*) malloc ( sizeof (Struct Node));
  newnode -> data = data;
  newnode -> next = head;
  head = newnode;
  printf (" Node inserted at the beginning.\n");
}
Void insertAtEnd (int data) {
  Struct Node *newnode = (Struct Node*) malloc (sizeof(Struct Node));
  newnode-> data =data;
  newnode -> next = NULL;
  if (head ==Null) {
        head = newnode;
  } else{
      Struct Node *temp = head;
      while (temp -> next != NULL)
            temp = temp ->next;
      temp -> next = newnode;
  }
printf ("Node inserted at the end.\n");
}
Void insertAtPosition (int data, int pos ) {
    int i;
    Struct Node *newnode, *temp=head;
    if (pos<1 ) {
        printf ("Invalid position.\n");
        return;
    }
    if (pos==1) {
        insertAtBeginning (data);
        return;
    }
    newnode = (Struct Node*) malloc(sizeof(Struct Node));
```

```
    newnode->data = data;
    for (i=1; i<pos-1 && temp!=NULL; i++)
        temp = temp->next;
    if (temp==NULL) {
        printf("Position out of range.\n");
        free(Newnode);
    } else {
        newnode->next = temp->next;
        temp->next = newnode;
        printf("Node inserted at position %d.\n", pos);
    }
}

void displayList() {
    struct Node *temp = head;
    if (head==NULL) {
        printf("List is empty.\n");
        return;
    }
    printf("\nLinked List: ");
    while (temp!=NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, n, data, pos;
    while (1) {
    printf("\n-- Singly Linked List operations --\n 1. create linked list \n 2. Insert at beginning \n 3. Insert at any position \n 4. Insert at end \n 5. display list \n 6. Exit\n");
    printf("Enter your choice:");
    scanf("%d", &choice);
    switch(choice) {
    case 1:
        printf("Enter number of nodes:");
        scanf("%d", &n);
```

```
            createList(n);
            break;
        case 2:
            printf("Enter data to insert:");
            scanf("%d", &data);
            insertAtBeginning(data);
            break;
        case 3:
            printf("Enter data to insert and position:");
            scanf("%d %d", &data, &pos);
            insertAtPosition(data, Pos);
            break;
        case 4:
            printf("Enter data to insert:");
            scanf("%d", &data);
            insertAtEnd(data);
            break;
        case 5:
            displayList();
        case 6:
            printf("Exiting...\n");
            exit(0);
        default:
            printf("Invalid choice. Try again.\n");
    }
}
return 0;
}
```

output:

```
--singly linked list operations--
1. create linked list
2. insert at beginning
3. insert at position
4. insert at end
5. display list
6. Exit
```

Enter your choice : 1
Enter number of nodes : 3
Enter data for node 1 : 10
Enter data for node 2 : 20
Enter data for node 3 : 30
Linked list created successfully.

-- Singly linked list operations --
1. create linked list
2. insert at beginning
3. Insert at position
4. Insert at end
5. display list
6 Exit
Enter your choice : 2
Enter data to insert : 5
Node inserted at the beganning.

--Singly linked list operations--
1. Create linked list
2 Insert at beginning
3. Insert at position
4. Insert at end
5. display list
6. Exit
Enter your choice : 3
Enter data and position : 15 3
Node inserted at position 3.

-- singly linked list operations --
1. create linked list
2 Insert at beginning
3. Insert at position
4. Insert at end
5. Display list
6. Exit
Enter your choice : 4
Enter data to insert : 35
Node inserted at the end.

-- Singly linked list operations --

1. create linked list
2. Insert at beginning
3. Insert at position
4. Insert at end
5. display list
6. Exit

Enter your choice : 5

Linked list : 5 → 10 → 15 → 20 → 30 → 35 → NULL

-- Singly linked list operations --

1. create linked list
2. Insert at beginning
3. Insert at position
4. Insert at end
5. Display list
6. Exit

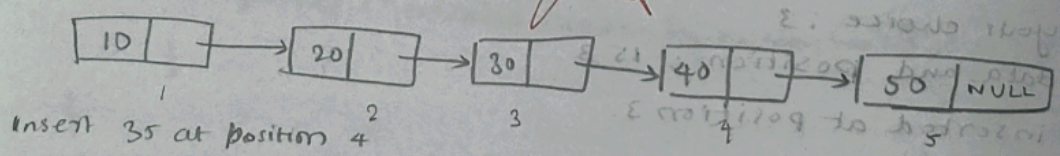
Enter your choice : 6

Exiting..

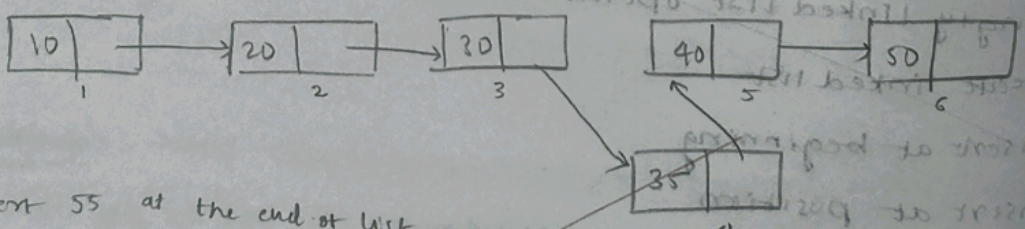
Trail:

CREATE LIST :

10 20 30 40 50



Insert 35 at position 4



Insert 55 at the end of list